

GoalCoach

An Agentic AI Learning Coach for Personalized Chinese Learning

1. The Proposed Application

GoalCoach is a multi-agent AI learning coach that helps learners master Chinese — with a focus on HSK exam preparation — through continuous, personalized planning rather than isolated conversations.

Existing Chinese-learning products fall into two categories. Structured platforms (e.g. HelloChinese, SuperChinese) offer systematic courses but fix the learning path and provide little real personalization beyond re-serving missed exercises. Conversational AI tutors (e.g. ChatGPT) are flexible but lack long-term planning: learners must know what to ask, progress is not tracked across sessions, and study easily becomes fragmented.

GoalCoach combines the strengths of both. Based on a learner's goal (e.g. pass HSK4, prepare for business Chinese), learning history and quiz performance, it continuously generates a dynamic, personalized daily study plan and proactively tells the learner what to study today — instead of waiting for a prompt or forcing a fixed curriculum.

Target users are Chinese-as-a-second-language learners, especially those preparing for HSK exams. The same architecture is designed to extend later to other languages, corporate training and other goal-based, long-term learning.

A key feature is a visible overall progress view: each day, the learner sees what percentage of their goal (e.g. HSK4 vocabulary and grammar) they have mastered **right now** — not a permanent ratchet, but a figure that honestly reflects both what has been learned and what is at risk of being forgotten (see Section 2). It is computed from LLM-graded practice rather than simple right/wrong scoring.

2. The Agentic System

GoalCoach is not a fixed pipeline that runs the same sequence of steps for every learner. State — the learner's goal, review schedule, plan validity and current session — determines which specialized agent runs next. The system is deliberately split into a deterministic control layer and a set of reasoning agents, rather than routing every decision through an LLM:

Workflow Orchestrator

Acts as a **lightweight, deterministic controller**, not an LLM-powered reasoning agent — routing does not need reasoning, so it does not spend a model call on it. On every visit it inspects the learner's state and dispatches to the appropriate agent by a fixed priority:

- 1. Goal just set or changed → Goal Planning Agent
- 2. A concept's scheduled review is due (per the Review Schedule below) → Progress & Mastery Agent
- 3. Daily plan invalid or exhausted → Goal Planning Agent
- 4. Otherwise (default) → Teaching Agent, to run today's plan

This keeps routing predictable, auditable and inexpensive, and leaves the model's reasoning budget for the steps that actually need judgment — the Progress and Teaching agents below.

Goal Planning Agent

Invoked only when a goal is set or changed; decomposes it into milestones and a roadmap. For a learner's very first session — before any error profile or mastery data exists — it also runs a short diagnostic assessment to seed initial mastery levels and starting milestones, so the Progress Agent is not routing from an empty state.

Progress & Mastery Agent

After each LLM-graded exercise, updates four learner-specific states:

- **Mastery Score** — how well a concept has been demonstrated when tested.
- **Retention Probability** — the estimated current chance the learner still remembers a concept, derived from a spaced-repetition-inspired forgetting model: it decays over time since last review, is strengthened and given a longer next interval on a good answer, and is reset on a poor one (in the spirit of interval-scheduling methods such as SM-2, without depending on a specific literature implementation).
- **Personal Error Profile** — the specific, recurring grammar and vocabulary mistakes that keep showing up across sessions (e.g. consistently confusing 了 vs. 过, or a specific set of measure words).
- **Next Review Schedule** — per-concept due dates driven by the retention model above.

Based on these four states, the agent autonomously decides whether the learner should advance to the next milestone, receive targeted remedial practice, or revisit a previously learned concept whose retention has dropped — not a hard-coded rule, but a decision grounded in LLM-graded performance plus the retention estimate.

It also computes overall progress as $\Sigma (\text{Mastery} \times \text{Retention})$ across the concepts in the learner's goal, rather than counting a concept as permanently mastered once passed. A concept learned a month ago and never reviewed contributes less to the total as its retention decays; revisiting it restores its weight. This is what keeps the headline progress percentage honest instead of a monotonically increasing counter.

Retrieval Agent

Called conditionally, only when the Progress Agent has flagged a specific gap or the plan requires new material — and once called, it is itself an autonomous decision point, not a pass-through to a vector search:

- Decides whether retrieval is actually needed, or whether previously retrieved material already covers the gap and can be reused.
- Formulates what to query for, prioritized toward the learner's own error profile and lowest-retention concepts rather than a generic weak-topic list.
- Re-ranks candidates returned from the vector database (ChromaDB) against the learner's specific error patterns before handing material to the Teaching Agent, rather than returning raw similarity results.

Teaching Agent

Generates open-ended, LLM-graded exercises (not multiple choice), explains concepts, runs practice dialogues, and adapts difficulty within the same session based on how the learner is answering. At the close of a session it produces a Session Summary (what was covered and how the learner performed) and seeds Tomorrow's Plan from the updated Review Schedule — closing the loop so the Workflow Orchestrator reads fresh state, not stale state, on the learner's next visit.

This differs from a standard chatbot — and from a simple prompt chain — in five concrete ways:

- (1) The Workflow Orchestrator performs genuine state-dependent branching on every visit — a real state machine, not a script that always executes the same order — while staying honest that this routing layer itself is deterministic, not an LLM making a judgment call.
- (2) The Progress Agent autonomously chooses among advance, remedial or revisit based on LLM-graded performance and estimated retention, not a fixed rule.
- (3) Grading open-ended answers and judging "mastery" requires LLM reasoning that a rule-based scoring system cannot reliably do.
- (4) Retrieval and Teaching are driven by each learner's own error profile and current retention gaps, not a generic weak-topic bucket shared across learners — and the Retrieval Agent itself decides whether, what and how to retrieve, rather than being invoked as a fixed lookup step.

- (5) Progress is not a permanent ratchet: it is continuously recomputed from mastery and retention together, so two learners with the same goal and the same nominal progress can see materially different numbers and materially different daily material, based on what each has actually retained versus forgotten.

None of this is scripted per learner — the reasoning-heavy decisions are made at run time from each learner's state; only the top-level routing between agents is intentionally kept deterministic.

3. The Technical Approach

We plan to orchestrate the workflow with LangGraph. The Workflow Orchestrator is implemented as conditional edges keyed on simple state flags (goal changed / review due / plan invalid) — no model call — while the Progress and Retrieval agents branch at run time on LLM-derived or model-estimated signals (advance vs. remedial vs. revisit; retrieve vs. reuse). Tool calling and structured outputs let agents exchange goals, plans, mastery scores, retention probability, the review schedule, the error profile and the overall progress percentage reliably. A vector database (ChromaDB) holds curated, HSK-aligned learning material for retrieval-augmented generation. The OpenAI API powers reasoning, LLM-based grading of open-ended exercises, and tutoring dialogue; FastAPI exposes the backend to a lightweight web frontend that displays the daily plan and the progress dashboard.

The retention model itself is intentionally simple: a per-concept decay function updated after every review outcome, tuned empirically rather than implemented against a specific spaced-repetition algorithm — enough to make progress and the review schedule behave sensibly without depending on unvalidated literature parameters.

Because the progress percentage and mastery decisions depend entirely on LLM grading, we treat grading reliability as a first-class technical problem rather than an assumption. Grading is done against explicit rubric dimensions (grammatical accuracy, semantic correctness, and vocabulary/register appropriateness) rather than a single pass/fail judgment, and a concept is only marked "mastered" after consistent performance across multiple independent exercises, not a single graded answer — this limits how much a single noisy or hallucinated grade can distort the reported progress. Before the grader is trusted inside the daily plan, we will validate it against a small hand-labeled set (roughly 20–30 answers per HSK level, human-graded on the same rubric) to measure agreement between the LLM grader and human graders, and log low-confidence or disagreeing cases for review rather than silently trusting every score.

The core technical challenge — and what we will build and test first — is the branching, state-driven control flow: reliably deciding when to advance, remedy, or revisit; keeping mastery, retention, the review schedule, the error profile and the progress percentage consistent across sessions; and grading free-form answers with an LLM in a way that is consistent enough to trust. Content breadth is secondary; the initial knowledge base will contain roughly 20–30 curated resources (vocabulary, grammar notes, short readings and dialogues at HSK3–5 level), enough to fully exercise the branching workflow. We will build and test this loop end-to-end in the first two weeks, then layer on the progress dashboard, Session Lifecycle UI and grader validation reporting in the remaining three.

4. The Value of the Application

Language learning is easy to start and hard to sustain: most learners abandon an app once a fixed course stops fitting their needs, or lose the discipline to keep prompting a chatbot themselves. GoalCoach removes the planning burden from the learner — it gives them one clear thing to do today, chosen specifically for them, based on their actual progress — and a visible, honestly-computed progress percentage that shows them they are moving toward their goal, a proven driver of retention in goal-based learning products.

Unlike fixed-curriculum apps, which route the same generic "review weak topics" content to every learner, and generic AI tutors, which keep no persistent record of a learner's mistakes at all, GoalCoach's Progress Agent maintains a personal error profile and a per-concept retention estimate, and feeds both directly into retrieval and teaching. Two learners preparing for the same HSK4 goal, even at the same nominal progress percentage, receive materially different daily material and see that percentage move differently over time, because each has a different history of mistakes and a different forgetting curve.

Rather than treating mastery as permanent, overall progress is continuously recalculated from both mastery and estimated retention: previously mastered concepts gradually contribute less to the overall progress if they are not reviewed, and regain their weight after successful revision. This is the concrete product differentiation: not "AI-generated planning" in the abstract, but a daily plan and a progress number that are both driven by each learner's own error history and forgetting pattern.

Because the agents separate goal planning, progress tracking, retrieval and teaching, the same architecture can be reused for other languages, corporate training or any subject with long-term, goal-based learning. AI is essential here, not incidental: only an LLM-driven agentic system can turn unstructured learning history, goals and a per-learner error and retention profile into a coherent, continuously updated plan at low cost — something static courses or rule-based systems cannot do.

5. The Team

Name	Role	Relevant Skills / Experience
[Weijia Han]	AI / Agent Engineer (team lead)	Project management experience at Bigo and Huawei, including user feedback analysis, coordinating data annotation, and cross-functional collaboration to support product optimization and contribute to model performance improvement. Bachelor's and Master's degrees in Translation, with coursework in Applied Linguistics. 42 Heilbronn student with hands-on experience in Python, Retrieval-Augmented Generation (RAG), and LLM-based application development, with a strong interest in agentic AI designs.
[Jiyan Wang]	Backend Engineer	42-Heilbronn-Studentin der Softwaretechnik, die eine solide Basis in Systemprogrammierung (C/C++), der Implementierung von HTTP-Servern (webserv), Prozesshandling (minishell) und Container-Infrastruktur (Inception) mit moderner Backend-Expertise in FastAPI, REST API design, databases und system integration verbindet.
[Weng Man Cheung]	AI / Data Engineer	Bachelor's degree in Translation, Master's degree in Lexicography, now student at 42 Heilbronn; hands-on experience working with LLMs and Python at 42 Heilbronn; completion of a 3-day duration AI agents workshop with Schwarz IT; strong linguistic and multilingual expertise; exposure to language acquisition research during studies; front-end HTML, CSS experience with online dictionaries; database management skills with language corpora, XML and XSLT.